

Assignment 4 evidence

YihaoYin (yihaoyin)

generated 2026-09-24 22:07 ????? on Windows-11-10.0.26200-SP0, polars 1.44.2

project HW4

evidence script sha256 76151d2e3fb565930aed34c3b78d3a4f8debede7fcdaae3d053cfd814d921ab8

Score **5.00 / 5** automatic, out of 6 for the assignment.

The last 1 point is REPORT.md, which your TA reads. This script does not score it.

Summary

group	score	checks	decided by
The table	1.00 / 1.00	4/4 passed	script
Baselines	1.00 / 1.00	4/4 passed	script
Models	1.25 / 1.25	5/5 passed	script
Honest evaluation	1.25 / 1.25	5/5 passed	script
Report numbers	0.50 / 0.50	2/2 passed	script
REPORT.md	? / 1.00	0/1 passed	your TA

automatic total	5.00 / 5.00	of 6 for the assignment	

What this script found

```
data      data\tep_fault_free_training.parquet
predictions results\predictions.parquet
forecast  results\forecast.csv
validation results\validation.csv
leaky     results\leaky.csv
report    REPORT.md
code      6 files
```

The table **1.00 / 1.00**

```
[PASS] predictions.parquet with the eight columns
[PASS] every test run and origin, for every horizon, and nothing else
[PASS] y_true is xmeas_18 at t + h
      100.00% of rows match
[PASS] shifts kept inside each run in the code
```

Baselines **1.00 / 1.00**

```
[PASS] forecast.csv with the five horizons and the six columns
[PASS] persistence RMSE matches the recomputation
      within 0.5 % at every horizon
[PASS] mean RMSE matches the recomputation
      mean over all samples of runs 1 to 300
[PASS] skill = 1 - direct / better baseline
```

Models **1.25 / 1.25**

```
[PASS] direct RMSE matches your predictions
[PASS] recursive RMSE matches your predictions
[PASS] direct beats the better baseline at four or more horizons
      5 of 5
[PASS] recursive differs from direct beyond h = 1
[PASS] a scaler inside a pipeline in the code
```

Honest evaluation **1.25 / 1.25**

```
[PASS] validation.csv with three or more settings
[PASS] validation on runs 301 to 400 in the code
[PASS] leaky.csv with both splits for runs 1 to 10
[PASS] TimeSeriesSplit with a gap of at least 10 in the code
      gap arguments found: ['H']
[PASS] shuffled KFold in the code
```

Report numbers 0.50 / 0.50

[PASS] RMSE values in REPORT.md come from forecast.csv
20 of 20 quoted; four needed
[PASS] REPORT.md quotes both leaky.csv means

REPORT.md 1.00 point, read by your TA

[SKIP] six sections and the AI-use line
read by your TA

Each line is decided from the output below, not asserted.

Transcript

data, and the script's own baselines

```
250,000 rows, 500 runs, samples per run [500]
training mean of xmeas_18 over runs 1 to 300: 65.80559
h= 1: persistence 0.0501, mean 0.4231 on 46,000 test origins
h= 5: persistence 0.1400, mean 0.4244 on 45,600 test origins
h=10: persistence 0.2218, mean 0.4260 on 45,100 test origins
h=20: persistence 0.3920, mean 0.4289 on 44,100 test origins
h=40: persistence 0.6353, mean 0.4345 on 42,100 test origins
```

predictions

```
222,900 rows, columns ['run', 't', 'h', 'y_true', 'persistence', 'mean', 'direct', 'recursive']
runs: 401 to 500, 100 distinct
h= 1: 46,000 rows (expected 46,000), 0 duplicated origins, 0 origins outside 40..499
h= 5: 45,600 rows (expected 45,600), 0 duplicated origins, 0 origins outside 40..495
h=10: 45,100 rows (expected 45,100), 0 duplicated origins, 0 origins outside 40..490
h=20: 44,100 rows (expected 44,100), 0 duplicated origins, 0 origins outside 40..480
h=40: 42,100 rows (expected 42,100), 0 duplicated origins, 0 origins outside 40..460
y_true equal to xmeas_18 at t + h: 100.00% of rows
```

forecast.csv against the script's recomputation

```
columns ['h', 'minutes', 'rows', 'persistence_rmse', 'mean_rmse', 'direct_rmse', 'recursive_rmse',
'skill', 'recursive_skill']
```

h	persistence yours / script	mean yours / script	direct yours / from preds	recursive yours / from preds
1	0.0501 / 0.0501	0.4231 / 0.4231	0.0367 / 0.0367	0.0393 / 0.0393
5	0.1400 / 0.1400	0.4244 / 0.4244	0.0869 / 0.0869	0.1070 / 0.1070
10	0.2218 / 0.2218	0.4260 / 0.4260	0.1271 / 0.1271	0.1486 / 0.1486
20	0.3920 / 0.3920	0.4289 / 0.4289	0.2029 / 0.2029	0.2482 / 0.2482
40	0.6353 / 0.6353	0.4345 / 0.4345	0.3328 / 0.3328	0.3846 / 0.3846

validation.csv

```
24 settings, columns ['h', 'n_lags', 'alpha', 'features', 'train_rows', 'train_rmse', 'val_rmse',
'val_persistence_rmse', 'chosen']
```

```
shape: (12, 9)
????????????????????????????????????????????????????????????????????????????????????
? h ? n_lags ? alpha ? features ? ? ? train_rmse ? val_rmse ? val_persistence_rmse ? chosen ?
? --- ? --- ? --- ? ? --- ? --- ? --- ? --- ?
? i64 ? i64 ? f64 ? i64 ? ? f64 ? f64 ? f64 ? bool ?
????????????????????????????????????????????????????????????????????????????????????
? 10 ? 1 ? 0.01 ? 12 ? ? ? 0.154756 ? 0.152825 ? 0.227914 ? false ?
? 10 ? 1 ? 1.0 ? 12 ? ? ? 0.154756 ? 0.152825 ? 0.227914 ? false ?
? 10 ? 1 ? 100.0 ? 12 ? ? ? 0.154768 ? 0.152833 ? 0.227914 ? false ?
? 10 ? 1 ? 10000.0 ? 12 ? ? ? 0.16248 ? 0.160314 ? 0.227914 ? false ?
? 10 ? 2 ? 0.01 ? 13 ? ? ? 0.149295 ? 0.14773 ? 0.227914 ? false ?
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? 10 ? 2 ? 10000.0 ? 13 ? ? ? 0.164293 ? 0.162113 ? 0.227914 ? false ?
```

```

? 10 ? 5 ? 0.01 ? 16 ? ? ? 0.143173 ? 0.142104 ? 0.227914 ? false ?
? 10 ? 5 ? 1.0 ? 16 ? ? ? 0.143173 ? 0.142103 ? 0.227914 ? false ?
? 10 ? 5 ? 100.0 ? 16 ? ? ? 0.143384 ? 0.142252 ? 0.227914 ? false ?
? 10 ? 5 ? 10000.0 ? 16 ? ? ? 0.161009 ? 0.159015 ? 0.227914 ? false ?
????????????????????????????????????????????????????????????????????????????????????

```

leaky.csv

```

20 rows, columns ['run', 'split', 'h', 'model_rmse', 'persistence_rmse']
runs with a shuffled row: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
runs with a time-ordered row: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```

```

shape: (2, 3)
????????????????????????????????????????????????????????????
? kind ? model_rmse ? persistence_rmse ?
? --- ? --- ? --- ?
? str ? f64 ? f64 ?
????????????????????????????????????????????????????????????
? shuffled ? 0.096044 ? 0.255007 ?
? time ? 0.344777 ? 0.251701 ?
????????????????????????????????????????????????????????????

```

read the code

```

6 code files: src\acf.py, src\forecast.py, src\leaky.py, src\pipeline.py, src\table.py, src\validate.py

shifts kept inside a run (.over / group_by / per-run loop): yes
scaler inside a pipeline: yes
validation runs 301 to 400 referenced: yes
TimeSeriesSplit: yes; gap arguments: ['H']
KFold with shuffle=True: yes

```

REPORT.md

```

1195 words, 7 headings: ['Report', '1. The table', '2. Baselines and skill', '3. Validation', '4. Direct
  against recursive', '5. Shuffle against time', '6. Limits']
RMSE values from forecast.csv quoted in the report: 20 of 20
leaky.csv means quoted: shuffled yes, time yes
AI-use line: yes

```

Code: src\acf.py

```

from __future__ import annotations

import numpy as np
import polars as pl

from table import DT_MIN, HORIZONS, RESULTS, SAMPLE, SAMPLES_PER_RUN, T_FIRST, TRAIN, Y, load, runs

MAX_LAG = 60

def autocorrelation(df: pl.DataFrame, max_lag: int = MAX_LAG) -> pl.DataFrame:
    """Autocorrelation of xmeas_18 in the training runs, pairs taken inside each run.

    Samples before T_FIRST are left out: there every run is still leaving the
    same initial steady state, and the forecasts are never scored there.
    For a stationary series persistence has MSE 2 var (1 - rho) and the mean
    has MSE var, so persistence_over_mean = sqrt(2 (1 - rho)) predicts the
    ratio of their RMSEs, and persistence wins only while rho > 0.5.
    """
    # One row per run, so a pair (s, s+k) never spans two runs.
    y = runs(df, TRAIN)[Y].to_numpy().reshape(-1, SAMPLES_PER_RUN)[: , T_FIRST - 1:]
    y = y - y.mean()
    var = np.mean(y ** 2)
    lags = np.arange(1, max_lag + 1)
    rho = np.array([np.mean(y[:, :-k] * y[:, k:]) / var for k in lags])

```

```

    return pl.DataFrame({"lag": lags, "minutes": lags * DT_MIN, "acf": rho,
                        "persistence_over_mean": np.sqrt(2 * (1 - rho))})

def main() -> pl.DataFrame:
    acf = autocorrelation(load())
    RESULTS.mkdir(exist_ok=True)
    acf.write_csv(RESULTS / "acf.csv")
    return acf

if __name__ == "__main__":
    acf = main()
    print(acf.filter(pl.col("lag").is_in(HORIZONS)))
    cross = acf.filter(pl.col("acf") < 0.5)["lag"].min()
    print(f"rho first drops below 0.5 at lag {cross} ({cross * DT_MIN} min)")

```

Code: src\forecast.py

```

from __future__ import annotations

import numpy as np
import polars as pl

from table import (DT_MIN, HORIZONS, RESULTS, RUN, TEST, TRAIN, build_table, load, ridge,
                  runs, train_mean, xy)
from validate import chosen

KEY = ["run", "t", "h"]
METHODS = ["persistence", "mean", "direct", "recursive"]

def origins(test: pl.DataFrame, h: int) -> pl.DataFrame:
    return test.select(pl.col(RUN).alias("run"), "t", pl.lit(h, dtype=pl.Int64).alias("h"))

def baselines(df: pl.DataFrame, mean: float) -> pl.DataFrame:
    """Persistence and the mean on every test origin, one row per (run, t, h)."""
    parts = []
    for h in HORIZONS:
        table, _ = build_table(runs(df, TEST), h, n_lags=1, valves=False)
        parts.append(origins(table, h).with_columns(
            y_true=table["target"],
            persistence=table["y[t]"],          # y[t+h] = y[t]
            mean=pl.lit(mean),                  # the training mean, whatever t is
        ))
    return pl.concat(parts)

def direct(df: pl.DataFrame, n_lags: int, alpha: float) -> tuple[pl.DataFrame, pl.DataFrame]:
    """One ridge model per horizon, lags of xmeas_18 plus the valves at t -> y[t+h].

    Also returns each model's coefficients on the standardized features, in
    deg C per standard deviation of the input, to show what it leans on.
    """
    parts, coefs = [], []
    for h in HORIZONS:
        table, names = build_table(df, h, n_lags, valves=True)
        model = ridge(alpha).fit(*xy(runs(table, TRAIN), names))
        test = runs(table, TEST)
        parts.append(origins(test, h).with_columns(
            direct=pl.Series(model.predict(test.select(names).to_numpy()))
        ))
        coefs.append(pl.DataFrame({"h": [h] * len(names), "feature": names,
                                   "coef": model[-1].coef_}))
    return pl.concat(parts), pl.concat(coefs)

```

```

def recursive(df: pl.DataFrame, n_lags: int, alpha: float) -> pl.DataFrame:
    """One model for y[t+1] from lags of xmeas_18 only, walked forward h steps.

    Each step's prediction becomes the new y[t] and the oldest lag drops off.
    The valves are left out because step two onwards would need the valves at
    t+1, t+2, ..., which are not known at t. The path from an origin does not
    depend on h, so it is walked once to the longest horizon, and read off at
    each horizon on the way.
    """
    table, names = build_table(df, 1, n_lags, valves=False)
    one_step = ridge(alpha).fit(*xy(runs(table, TRAIN), names))
    test = runs(table, TEST) # origins 40..499, every horizon's origins
    lags = test.select(names).to_numpy()
    parts = []
    for step in range(1, max(HORIZONS) + 1):
        nxt = one_step.predict(lags)
        lags = np.column_stack([nxt, lags[:, :-1]])
        if step in HORIZONS:
            parts.append(origins(test, step).with_columns(recursive=pl.Series(nxt)))
    return pl.concat(parts)

def score(pred: pl.DataFrame, methods: list[str]) -> pl.DataFrame:
    """RMSE of each method at each horizon, over the same test origins."""
    return (pred.group_by("h")
            .agg(pl.len().alias("rows"),
                 *[(pl.col(m) - pl.col("y_true")) ** 2].mean().sqrt().alias(f"{m}_rmse")
                   for m in methods])
            .sort("h"))

def main() -> pl.DataFrame:
    df = load()
    n_lags, alpha = chosen() # picked on runs 301 to 400 by validate.py
    direct_pred, coefs = direct(df, n_lags, alpha)
    # The baselines fix the rows: every test origin t = 40 .. 500-h, for each h.
    pred = (baselines(df, train_mean(df))
            .join(direct_pred, on=KEY, how="left")
            .join(recursive(df, n_lags, alpha), on=KEY, how="left")
            .select(*KEY, "y_true", *METHODS)
            .sort("h", "run", "t"))
    if pred.null_count().sum_horizontal().item():
        raise ValueError("a test origin is missing a forecast")

    better = pl.min_horizontal("persistence_rmse", "mean_rmse")
    table = (score(pred, METHODS)
            .with_columns(minutes=pl.col("h") * DT_MIN,
                          skill=1 - pl.col("direct_rmse") / better,
                          recursive_skill=1 - pl.col("recursive_rmse") / better)
            .select("h", "minutes", "rows", *[f"{m}_rmse" for m in METHODS],
                    "skill", "recursive_skill"))
    RESULTS.mkdir(exist_ok=True)
    pred.write_parquet(RESULTS / "predictions.parquet")
    table.write_csv(RESULTS / "forecast.csv")
    coefs.write_csv(RESULTS / "coefficients.csv")
    return table

if __name__ == "__main__":
    print("chosen (n_lags, alpha):", chosen())
    with pl.Config(float_precision=4):
        print(main())

```

Code: src\leaky.py

```
from __future__ import annotations
```

```

import numpy as np
import polars as pl
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold, TimeSeriesSplit

from table import RESULTS, RUN, build_table, load, rmse, xy
from validate import chosen

H = 10
RUNS = range(1, 11)

def splits() -> dict:
    return {
        # Rows go to train and test at random, so each test row's neighbours,
        # a few minutes before and after, are usually in the training set.
        "shuffled": KFold(n_splits=5, shuffle=True, random_state=0),
        # Train on the past, skip H origins, test on what follows. A training
        # row at origin t has target y[t+H]; with gap=H the last one still
        # lands before the first test origin, so no training target is taken
        # from the test stretch.
        "time": TimeSeriesSplit(n_splits=5, gap=H),
    }

def score_run(df: pl.DataFrame, run: int, n_lags: int) -> list[dict]:
    """One run on its own: a forest and persistence, scored on the same test folds."""
    table, names = build_table(df.filter(pl.col(RUN) == run), H, n_lags) # direct features
    X, y = xy(table, names) # rows in time order, column 0 is y[t]
    rows = []
    for name, cv in splits().items():
        forest, persistence = [], []
        for train, test in cv.split(X):
            # One thread: parallel trees are summed in a varying order, which
            # changes the last digit of the RMSE from one run to the next.
            model = RandomForestRegressor(n_estimators=100, random_state=0, n_jobs=1)
            model.fit(X[train], y[train])
            forest.append(rmse(y[test] - model.predict(X[test])))
            persistence.append(rmse(y[test] - X[test, 0]))
        rows.append({"run": run, "split": name, "h": H,
                    "model_rmse": float(np.mean(forest)),
                    "persistence_rmse": float(np.mean(persistence))})
    return rows

def main() -> pl.DataFrame:
    df = load()
    n_lags, _ = chosen()
    out = pl.DataFrame([row for run in RUNS for row in score_run(df, run, n_lags)])
    RESULTS.mkdir(exist_ok=True)
    out.write_csv(RESULTS / "leaky.csv")
    return out

if __name__ == "__main__":
    out = main()
    with pl.Config(float_precision=4, tbl_rows=30):
        print(out)
        print(out.group_by("split").agg(pl.col("model_rmse", "persistence_rmse").mean()).sort("split"))

```

Code: src\pipeline.py

```

from __future__ import annotations

import acf
import forecast
import leaky

```

```

import validate

def run() -> None:
    a = acf.main()
    cross = a.filter(a["acf"] < 0.5)["lag"].min()
    print(f"acf          rho first below 0.5 at lag {cross}")

    v = validate.main()
    n_lags, alpha = validate.chosen()
    print(f"validate      {v.height} settings on runs 301 to 400; chosen {n_lags} lags, alpha {alpha}")

    f = forecast.main()
    wins = int((f["skill"] > 0).sum())
    print(f"forecast      direct beats the better baseline at {wins} of {f.height} horizons")

    k = leaky.main().group_by("split").agg(leaky.pl.col("model_rmse").mean()).sort("split")
    print(f"leaky          mean forest RMSE " + ", ".join(f"{s} {m:.4f}" for s, m in k.iter_rows()))

if __name__ == "__main__":
    run()

```

Code: src\table.py

```

from __future__ import annotations

from pathlib import Path

import numpy as np
import polars as pl
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

ROOT = Path(__file__).resolve().parent.parent
DATA = ROOT / "data" / "tep_fault_free_training.parquet"
RESULTS = ROOT / "results"

Y = "xmeas_18"                # stripper temperature, deg C
RUN = "simulationRun"
SAMPLE = "sample"
XMV = [f"xmv_{i}" for i in range(1, 12)]
DT_MIN = 3                    # minutes between samples

# The fixed choices of the assignment.
HORIZONS = [1, 5, 10, 20, 40]
TRAIN = (1, 300)
VALIDATION = (301, 400)      # hyperparameters only
TEST = (401, 500)            # touched once, by forecast.py
T_FIRST = 40                  # first forecast origin in every run
SAMPLES_PER_RUN = 500

def load() -> pl.DataFrame:
    """The fault-free training file, sorted, after checking every run is whole.

    A shift by k rows is a shift by k samples only if no run has a missing or
    repeated sample, so that is checked here rather than assumed.
    """
    df = pl.read_parquet(DATA).sort(RUN, SAMPLE)
    per_run = df.group_by(RUN).agg(n=pl.len(), first=pl.col(SAMPLE).min(),
                                   last=pl.col(SAMPLE).max(), unique=pl.col(SAMPLE).n_unique())
    bad = per_run.filter((pl.col("n") != SAMPLES_PER_RUN) | (pl.col("unique") != SAMPLES_PER_RUN)
                          | (pl.col("first") != 1) | (pl.col("last") != SAMPLES_PER_RUN))
    if bad.height or per_run.height != 500:
        raise ValueError(f"expected 500 runs of samples 1..{SAMPLES_PER_RUN}:\n{bad}")

```

```

    return df

def runs(frame: pl.DataFrame, which: tuple[int, int]) -> pl.DataFrame:
    return frame.filter(pl.col(RUN).is_between(*which))

def lag_names(n_lags: int) -> list[str]:
    """y[t] first, so column 0 of every feature matrix is the persistence forecast."""
    return ["y[t]"] + [f"y[t-{k}]" for k in range(1, n_lags)]

def build_table(df: pl.DataFrame, h: int, n_lags: int, valves: bool = True) -> tuple[pl.DataFrame, list[str]]:
    """One row per (run, t): the features known at t, and the target y[t+h].

    Lag k is xmeas_18 at sample t-k, for k = 0 .. n_lags-1; the valves are the
    ones at sample t itself, because later valve moves are not known when the
    forecast is made. Every shift runs over(RUN), ordered by sample, so no lag
    and no target reaches into a neighbouring run: at the edges of a run the
    shift gives null and the row is dropped. Origins then start at T_FIRST, so
    every lag depth up to 40 is fitted and scored on the same (run, t) rows.
    """
    if not 1 <= n_lags <= T_FIRST:
        raise ValueError(f"lag depth must be 1..{T_FIRST}, got {n_lags}")
    names = lag_names(n_lags)
    features = {name: pl.col(Y).shift(k).over(RUN, order_by=SAMPLE)
                for k, name in enumerate(names)}
    if valves:
        features |= {v: pl.col(v) for v in XMV}
    table = (df.select(RUN, pl.col(SAMPLE).alias("t"), **features,
                    target=pl.col(Y).shift(-h).over(RUN, order_by=SAMPLE))
            .drop_nulls()
            .filter(pl.col("t") >= T_FIRST)
            .sort(RUN, "t"))
    return table, list(features)

def xy(table: pl.DataFrame, names: list[str]) -> tuple[np.ndarray, np.ndarray]:
    return table.select(names).to_numpy(), table["target"].to_numpy()

def train_mean(df: pl.DataFrame) -> float:
    """The mean forecast: xmeas_18 over all samples of the training runs."""
    return float(runs(df, TRAIN)[Y].mean())

def ridge(alpha: float):
    # The scaler sits inside the pipeline, so it is fitted on training rows only.
    return make_pipeline(StandardScaler(), Ridge(alpha=alpha))

def rmse(error) -> float:
    return float(np.sqrt(np.mean(np.square(np.asarray(error)))))

if __name__ == "__main__":
    df = load()
    for h in HORIZONS:
        table, names = build_table(runs(df, TEST), h, n_lags=10)
        print(f"h={h:>2}: {table.height:,} test rows, origins {table['t'].min()}..{table['t'].max()}")
        print(table.select(RUN, "t", *names[:3], "xmv_1", "target").head(3))

```

Code: src\validate.py

```
from __future__ import annotations
```

```

import polars as pl

from table import RESULTS, TRAIN, VALIDATION, build_table, load, ridge, rmse, runs, xy

H = 10                                     # hyperparameters are chosen at 30 minutes
LAGS = [1, 2, 5, 10, 20, 40]
ALPHAS = [0.01, 1.0, 100.0, 10_000.0]

def sweep(df: pl.DataFrame) -> pl.DataFrame:
    """Fit on runs 1 to 300, score on runs 301 to 400, for every (lag depth, alpha).

    Only runs 1 to 400 are handed to this function: the test runs are not
    loaded here at all, so they cannot play any part in the choice.
    """
    rows = []
    for n_lags in LAGS:
        table, names = build_table(df, H, n_lags)          # lags + valves at t
        X_train, y_train = xy(runs(table, TRAIN), names)
        X_val, y_val = xy(runs(table, VALIDATION), names)
        for alpha in ALPHAS:
            model = ridge(alpha).fit(X_train, y_train)
            rows.append({"h": H, "n_lags": n_lags, "alpha": alpha, "features": len(names),
                        "train_rows": len(y_train),
                        "train_rmse": rmse(y_train - model.predict(X_train)),
                        "val_rmse": rmse(y_val - model.predict(X_val)),
                        "val_persistence_rmse": rmse(y_val - X_val[:, 0])})
    out = pl.DataFrame(rows)
    return out.with_columns(chosen=pl.col("val_rmse") == pl.col("val_rmse").min())

def chosen() -> tuple[int, float]:
    """The setting with the lowest validation RMSE, read back from validation.csv."""
    best = pl.read_csv(RESULTS / "validation.csv").filter(pl.col("chosen")).row(0, named=True)
    return int(best["n_lags"]), float(best["alpha"])

def main() -> pl.DataFrame:
    seen = runs(load(), (TRAIN[0], VALIDATION[1]))          # runs 1 to 400 only
    out = sweep(seen)
    RESULTS.mkdir(exist_ok=True)
    out.write_csv(RESULTS / "validation.csv")
    return out

if __name__ == "__main__":
    out = main()
    print(out.sort("n_lags", "alpha"))
    print("chosen (n_lags, alpha):", chosen())

```

REPORT: REPORT.md

```

# Report
Yihao Yin (yihaoyin)

```

Forecasting the Tennessee Eastman stripper temperature 'xmeas_18' (deg C) 1 to 40 samples (3 to 120 min) ahead. 'uv run python src/pipeline.py' rebuilds every file in 'results/', byte-identical on rerun.

1. The table

'build_table(df, h, n_lags)' in 'src/table.py' returns one row per (run, t): the lags $y[t]$, $y[t-1]$, ..., $y[t-n_lags+1]$ of 'xmeas_18', the valves 'xmv_1' to 'xmv_11' at sample t , and the target $y[t+h]$. All but the target is known at t ; the valves are taken at t only, since later moves are not known when the forecast is made. Every lag and the target is 'pl.col(Y).shift(k).over(RUN, order_by=SAMPLE)', so each shift restarts in each run and gives null at its edges instead of reading the neighbouring run. Those rows are dropped, and origins start at $t = 40$, so every lag depth up to 40 is scored on the same rows. 'load()' checks that every run holds samples 1 to 500 exactly once, so a shift by k rows is a shift by k samples. Checked against a per-run array, every lag, valve and target is 'xmeas_18' at $t-k$, t and $t+h$

of the same run.

2. Baselines and skill

'results/forecast.csv', test runs 401 to 500, RMSE in deg C; skill = 1 - direct / min(persistence, mean).

h	min	persistence	mean	direct	recursive	skill
1	3	0.0501	0.4231	0.0367	0.0393	0.2680
5	15	0.1400	0.4244	0.0869	0.1070	0.3791
10	30	0.2218	0.4260	0.1271	0.1486	0.4268
20	60	0.3920	0.4289	0.2029	0.2482	0.4825
40	120	0.6353	0.4345	0.3328	0.3846	0.2340

Persistence beats the mean (65.8056 deg C) up to h = 20 (0.3920 against 0.4289) and loses at h = 40 (0.6353 against 0.4345). This matches the autocorrelation of 'xmeas_18' in the training runs ('results/acf.csv'): rho is 0.868 at 10 samples and 0.572 at 20, falls below 0.5 at lag 23 (69 min), and is -0.155 at 40. The temperature oscillates, so two hours on it tends to be across the mean and holding today's value points the wrong way. Direct beats the better baseline at all five horizons; skill peaks at one hour, where neither free forecast is good.

3. Validation

'make_pipeline(StandardScaler(), Ridge(alpha))' at h = 10 on lags plus valves, fitted on runs 1 to 300 and scored on runs 301 to 400; 'src/validate.py' never loads the test runs. Lag depths 1, 2, 5, 10, 20, 40 and alpha 0.01, 1, 100, 10000 give 24 settings ('results/validation.csv'). The lowest validation RMSE, 0.1280 against 0.2279 for persistence, is 40 lags with alpha = 1. Lag depth mattered most: 0.1528 with one lag, 0.1308 with 10, 0.1280 with 20; 40 beat 20 by 0.00005, a tie in practice. A history shows which way the temperature is moving; one value cannot. Alpha from 0.01 to 100 changed only the fourth decimal, since 135,300 training rows hold 51 coefficients in check, and 10000 hurt (0.1364) by shrinking useful ones. Train and validation RMSE agree (0.1277 against 0.1280): nothing to overfit.

4. Direct against recursive

Direct is one ridge per horizon on 40 lags plus the valves at t. Recursive is one ridge for y[t+1] on the 40 lags alone, applied h times with each prediction shifted into y[t]. Direct wins at every horizon: 0.0367 against 0.0393 at h = 1, 0.1271 against 0.1486 at h = 10, 0.3328 against 0.3846 at h = 40. At h = 1 nothing compounds, so that gap is the valves alone, mostly the stripper steam valve 'xmv_9', which shows where the temperature is being pushed before it moves. The gap then grows from 0.0026 to 0.0518 deg C: the recursive model was trained for one step and feeds its own errors back, while each direct model is fitted to its own horizon. Recursive still beats the mean at two hours; 40 lags describe the oscillation, and a stable linear recursion relaxes toward the mean rather than diverging. It has no valves because from the second step on it would need the valves at t+1, t+2, ..., which are unknown at t: holding them fixed assumes the controllers stop, and forecasting them needs eleven more models.

5. Shuffle against time

Runs 1 to 10 one at a time, h = 10, the direct features, a 100-tree random forest, 5 folds, persistence on the same test folds. Means over the runs of 'results/leaky.csv':

split	forest	persistence
'KFold(n_splits=5, shuffle=True)'	0.0960	0.2550
'TimeSeriesSplit(n_splits=5, gap=10)'	0.3448	0.2517

Shuffled, the forest looks 62% better than persistence; in time order it is 37% worse, and both hold in every one of the ten runs. I would report 0.3448 to a plant manager: in use the model always forecasts a future it has not seen, which is what the time split measures. Shuffled, each test row's neighbours a few minutes either side are in the training set, and their targets y[t+h-1] and y[t+h+1] are almost the answer (rho is 0.994 at one lag), so the forest interpolates between answers it has seen. With about 360 rows of one run it even appears to beat the ridge model trained on 300 runs (0.1271), which only a leak allows. The gap must be at least h because a training row at origin t has target y[t+h]; with a smaller gap the last training targets fall inside the test stretch. In time order the forest loses to persistence because it has only 66 to 366 training rows and cannot predict outside the range of targets it has seen.

6. Limits

The model has only seen fault-free runs, so its errors are known only for normal data, and it leans on one input: at h = 10 the steam valve 'xmv_9' carries +0.30 deg C per standard deviation ('results/coefficients.csv'), more than three times the next valve ('xmv_5', -0.09). Suppose the steam supply pressure drops. 'xmv_9' still reads the same opening, but less steam passes through it and the

stripper cools. Having only ever seen the temperature follow that valve, the model keeps forecasting the temperature this opening has always given. Nothing warns it: every input is in its usual range, the output is one number, and the 0.1271 deg C it scored at 30 minutes describes a plant that behaves like the fault-free simulations, not this one. It needs a separate check, such as an alarm on its own one-step residuals, to say when its training no longer applies.

Generative AI use: I used Claude to help me with the code debug; every number in it is computed by the code in 'src/' and saved in 'results/'.

Summary line

The script hash should match the checksum published beside the script.

```
{"andrew_id": "yihaoyin", "generated": "2026-09-24 22:07 \u4e1c\u90e8\u590f\u4ee4\u65f6", "auto_score": 5.0, "auto_of": 5.0, "report_points": 1.0, "held_for_ta": 0.0, "assignment_total": 6.0, "groups": {"table": 1.0, "baseline": 1.0, "models": 1.25, "honest": 1.25, "numbers": 0.5}, "script_sha256": "76151d2e3fb565930aed34c3b78d3a4f8debede7fcdcae3d053cfd814d921ab8"}
```